

Δεύτερο Σετ Ασκήσεων
PART II: “BSTs (Balanced Search Trees) &&
HASHING”
Δομές Δεδομένων 2025-2026

Ανδριόπουλος Ιωάννης AM: 1115519
Καραπατάκης Ευθύμιος AM: 1115521
Χριστοπούλου Αγγελική AM: 1115471

19 Ιουνίου 2026

Περιεχόμενα

1	Εκφώνηση Εργασίας	3
1.1	Ζητούμενα	3
2	Εισαγωγή	5
3	Περιγραφή Δεδομένων	5
4	Επισκόπηση Υλοποίησης	6
4.1	Φόρτωση Δεδομένων	6
5	Άσκηση (Α): Υλοποίηση Εφαρμογής με BST, με βάση DATE	7
5.1	Περιγραφή	7
6	Άσκηση (Β): Υλοποίηση Εφαρμογής με BST, με βάση CUMULATIVE	9
6.1	Περιγραφή	9
7	Άσκηση (Γ): Υλοποίηση Εφαρμογής με Hashing με αλυσίδες, με βάση DATE	11
7.1	Περιγραφή	11
8	Συμπεράσματα	14
8.1	Ταξινόμηση και Εύρος	14
8.2	Ταχύτητα Αναζήτησης Συγκεκριμένου Κλειδιού	14
8.3	Μοναδικότητα Δεδομένων	14
9	Παράρτημα	15
9.1	Πηγαίος Κώδικας	15
9.2	Στιγμιότυπα Οθόνης	15

1 Εκφώνηση Εργασίας

Με τον κατάλληλο ορισμό δομών (structs) / κλάσεων (classes) και συναρτήσεων (functions) / μεθόδων (methods), να υλοποιήσετε μια εφαρμογή (σε γλώσσα C, C++ ή Java) που θα επεξεργάζεται τα δεδομένα του αρχείου “dataset effects-of-covid-19-on-trade-at-15-december-2021-provisional.csv”. Θυμίζουμε ξανά ότι κάθε γραμμή του αρχείου αυτού αντιστοιχεί σε μία ημέρα μετρήσεων.

1.1 Ζητούμενα

- (Α) Η εφαρμογή διαβάζει αρχικά το αρχείο και δημιουργεί ένα Δένδρο BST (Ισοζυγισμένο Δυαδικό Δέντρο) στο οποίο κάθε κόμβος του διατηρεί την εγγραφή (Date, Cumulative της ημέρας αυτής). Το BST διατάσσεται ως προς την ΗΜΕΡΟΜΗΝΙΑ (Date) και υλοποιείται με δυναμική διαχείριση μνήμης. Προτιμότερες επιλογές AVL, red-black, (2,4) δέντρα. Μετά την δημιουργία του BST η εφαρμογή εμφανίζει ένα μενού με τις ακόλουθες επιλογές:

1. Απεικόνιση του BST με ενδο-διατεταγμένη διάσχιση. Κάθε απεικόνιση θα πρέπει να περιέχει μια επικεφαλίδα με τους τίτλους των στοιχείων των εγγραφών που απεικονίζονται.
2. Αναζήτηση της τιμής Cumulative βάσει ΗΜΕΡΟΜΗΝΙΑΣ (Date) που θα δίνεται από το χρήστη.
3. Τροποποίηση του περιεχομένου του πεδίου Cumulative που αντιστοιχεί σε συγκεκριμένη ΗΜΕΡΟΜΗΝΙΑ (Date).
4. Διαγραφή μιας εγγραφής που αντιστοιχεί σε συγκεκριμένη ΗΜΕΡΟΜΗΝΙΑ (Date).
5. Έξοδος από την εφαρμογή.

- (Β) Τροποποιήστε κατάλληλα τον κώδικα του (Α), ώστε το αρχείο να διαβάζεται στο BST με βάση την τιμή του πεδίου Cumulative. Το BST διατάσσεται ως προς την τιμή Cumulative ανά ΗΜΕΡΟΜΗΝΙΑ (Date) και υλοποιείται με δυναμική διαχείριση μνήμης. Μετά την δημιουργία του BST η εφαρμογή εμφανίζει ένα μενού με τις ακόλουθες επιλογές:

1. Εύρεση ΗΜΕΡΑΣ/ΗΜΕΡΩΝ με την ΕΛΑΧΙΣΤΗ ΤΙΜΗ Cumulative.
2. Εύρεση ΗΜΕΡΑΣ/ΗΜΕΡΩΝ με τη ΜΕΓΙΣΤΗ ΤΙΜΗ Cumulative.

- (Γ) Υλοποιήστε το (Α) κάνοντας χρήση HASHING με αλυσίδες, αντί BST. Η συνάρτηση κατακερματισμού θα υπολογίζεται ως το υπόλοιπο (modulo) της διαίρεσης του αθροίσματος των κωδικών ASCII των επιμέρους χαρακτήρων που απαρτίζουν την ΗΜΕΡΟΜΗΝΙΑ με ένα περιττό αριθμό m που συμβολίζει το πλήθος των κάδων (buckets). Π.χ. για ΗΜΕΡΟΜΗΝΙΑ=" 15/12/2021" και m=11, ισχύει:

$$\text{Hash}("15/12/2021") = [\text{ASCII}('1') + \text{ASCII}('5') + \text{ASCII}('/') + \text{ASCII}('1') + \text{ASCII}('2') + \text{ASCII}('/') + \text{ASCII}('2') + \text{ASCII}('0') + \text{ASCII}('2') + \text{ASCII}('1')] \bmod 11.$$

Το πρόγραμμα θα εμφανίζει ένα μενού με τις ακόλουθες επιλογές:

1. Αναζήτηση Τιμής Cumulative βάσει της ΗΜΕΡΟΜΗΝΙΑΣ που θα δίνεται από το χρήστη.

2. Τροποποίηση των στοιχείων εγγραφής βάσει ΗΜΕΡΟΜΗΝΙΑΣ που θα δίνεται από το χρήστη. Η τροποποίηση προφανώς αφορά ΜΟΝΟ το πεδίο Cumulative.
3. Διαγραφή μιας εγγραφής από τον πίνακα κατακερματισμού βάσει ΗΜΕΡΟΜΗΝΙΑΣ που θα δίνεται από το χρήστη.
4. Έξοδος από την εφαρμογή.

Ενοποιήστε τα (Α), (Β) και (Γ) σε ένα πρόγραμμα στο οποίο ο χρήστης θα ερωτάται αν θέλει τη φόρτωση του αρχείου σε ένα BST ή σε μία δομή Hashing με αλυσίδες και στην περίπτωση που ο χρήστης επιλέξει το πρώτο να μπορεί εν συνεχεία να επιλέξει αν η φόρτωση στο BST θα γίνει με βάση την ΗΜΕΡΟΜΗΝΙΑ ή την τιμή Cumulative ανά ημέρα.

2 Εισαγωγή

Αυτό το project αναλύει το dataset *Effects of COVID-19 on Trade (2015–2021)* το οποίο περιέχει 111.438 εγγραφές. Κάθε εγγραφή περιέχει:

Direction, Year, Date, Weekday, Country, Commodity, Transport Mode, Measure, Value, Cumulative

Στόχος είναι η υλοποίηση και η σύγκριση κλασικών αλγορίθμων ταξινόμησης και αναζήτησης χρησιμοποιώντας αυτό το πραγματικό σύνολο δεδομένων.

3 Περιγραφή Δεδομένων

Το dataset περιέχει στατιστικά στοιχεία εμπορίου που σχετίζονται με εισαγωγές, εξαγωγές και επανεισαγωγές. Κάθε εγγραφή περιλαμβάνει αριθμητικά πεδία όπως:

- **Value:** Εμπορική αξία σε δολάρια ή τόνους
- **Cumulative:** Αθροιστική εμπορική αξία σε δολάρια ή τόνους
- **Date:** Ημερομηνία στην μορφή HH/MM/YYYY

Πεδίο	Τύπος	Περιγραφή
Direction	String	imports, exports, reimports
Year	Integer	2015-2021
Date	Date	dd/mm/yyyy
Weekday	String	Monday-Sunday
Country	String	Χώρα εμπορικού εταίρου
Commodity	String	Κατηγορία προϊόντος
Transport_Mode	String	sea, air, κλπ.
Measure	String	\$ ή Tonnes
Value	Long Integer	Εμπορική αξία
Cumulative	Long Integer	Αθροιστική αξία

Πίνακας 1: Δομή του Dataset

4 Επισκόπηση Υλοποίησης

Το σύστημα υλοποιήθηκε σε C και περιλαμβάνει 2 βασικές ενότητες:

- Δημιουργία Δέντρου BST
- Δημιουργία Hashing με αλυσίδες

```
1 typedef struct {
2     char direction[DIRECTION];
3     char year[YEAR];
4     char date[DATE];
5     char weekday[WEEKDAY];
6     char country[MAX_STR];
7     char commodity[MAX_STR];
8     char transport_mode[MAX_STR];
9     char measure[MAX_STR];
10    long long value;
11    long long cumulative;
12 } Record;
```

Listing 1: Δομή Record στη C

4.1 Φόρτωση Δεδομένων

```
1 int load_csv(const char *filename, Record *data) {
2     FILE *fptr = fopen(filename, "r");
3     if(!fptr) {
4         printf("[ERROR] No file %s found.\n", filename);
5         return -1;
6     }
7
8     char line[512]; //temp info saver
9     int count = 0;
10
11    fgets(line, sizeof(line), fptr);
12
13    while (fgets(line, sizeof(line), fptr) && count < MAX_ROWS) {
14        line[strcspn(line, "\r\n")] = 0;
15
16        char *temp;
17        Record r;
18        memset(&r, 0, sizeof(r));
19
20        temp = strtok(line, ",");
21        if(!temp) continue;
22        strncpy(r.direction, temp, DIRECTION-1);
23
24        temp = strtok(NULL, ",");
25        if(!temp) continue;
26        strncpy(r.year, temp, YEAR-1);
27
28        temp = strtok(NULL, ",");
29        if(!temp) continue;
30        strncpy(r.date, temp, DATE-1);
```

```

31     temp = strtok(NULL, ",");
32     if(!temp) continue;
33     strncpy(r.weekday, temp, sizeof(r.weekday) - 1);
34
35     temp = strtok(NULL, ",");
36     if(!temp) continue;
37     strncpy(r.country, temp, sizeof(r.country) - 1);
38
39     temp = strtok(NULL, ",");
40     if(!temp) continue;
41     strncpy(r.commodity, temp, sizeof(r.commodity) - 1);
42
43     temp = strtok(NULL, ",");
44     if(!temp) continue;
45     strncpy(r.transport_mode, temp, sizeof(r.transport_mode) - 1);
46
47     temp = strtok(NULL, ",");
48     if(!temp) continue;
49     strncpy(r.measure, temp, sizeof(r.measure) - 1);
50
51     temp = strtok(NULL, ",");
52     if(!temp) continue;
53     r.value = atoll(temp);
54
55     temp = strtok(NULL, ",");
56     if(!temp) continue;
57     r.cumulative = atoll(temp);
58
59     data[count++] = r;
60 }
61
62 fclose(fp);
63 return count;
64 }
65

```

Listing 2: Φόρτωση CSV

5 Άσκηση (Α): Υλοποίηση Εφαρμογής με BST, με βάση DATE

5.1 Περιγραφή

Υλοποιήθηκε με Δέντρο AVL. Κάθε κόμβος του δέντρου αποθηκεύει έναν δείκτη στον δυναμικά δεσμευμένο πίνακα που περιέχει τα δεδομένα του CSV.

```

1     typedef struct Node {
2         Record* record;
3         struct Node* left;
4         struct Node* right;
5         int height;
6     } Node;

```

Listing 3: Δομή κόμβου

1. Για το πρώτο ερώτημα, το δέντρο εμφανίζεται με in-order Traversal.

```

1 void inorder(Node* root) {
2     if(!root) return;
3
4     inorder(root->left);
5
6     printf("Date: %-d/%d/%d \t Cumulative: %lld\n",
7         root->record->date[0], root->record->date[1], root->record->date[2],
8         root->record->cumulative);
9
10    inorder(root->right);
11 }

```

Listing 4: Συνάρτηση inorder

2. Για το δεύτερο ερώτημα, χρησιμοποιείται κλασική Δυναδική Αναζήτηση. Η δοθείσα ημερομηνία συγκρίνεται με τον τρέχοντα κόμβο. Αν είναι μικρότερη, η αναζήτηση συνεχίζεται στο αριστερό παιδί. Αν είναι μεγαλύτερη, στο δεξί.

```

1 Node* search(Node* root, int date[3]) {
2     if(!root) return NULL;
3
4     int cmp = compareDates(date, root->record->date);
5
6     if(cmp == 0) return root;
7
8     if(cmp < 0) return search(root->left, date);
9
10    return search(root->right, date);
11 }

```

Listing 5: Συνάρτηση search

3. Για το τρίτο ερώτημα, χρησιμοποιείται η συνάρτηση search του προηγούμενου ερωτήματος και όταν βρεθεί ο κόμβος, ο αλγόριθμος τον αντικαθιστά με την νέα τιμή.

```

1 Node* modify(Node* root, int date[3], long long value) {
2     Node* node = search(root, date);
3
4     if (node) {
5         node->record->cumulative = value;
6     } else {
7         printf("Date not found.\n");
8     }
9
10    return root;
11 }

```

Listing 6: Συνάρτηση modify

4. Για το τέταρτο ερώτημα, χρησιμοποιείται η συνάρτηση search του προηγούμενου ερωτήματος και αν βρεθεί διαγράφεται. Αν έχει δύο παιδιά, δεν κόβονται οι συνδέσεις του δέντρου. Αντίθετα, βρίσκεται ο "in-order διάδοχος", ανταλλάσσονται οι δείκτες Record* (swapNodes), και διαγράφεται αναδρομικά ο διπλότυπος κόμβος. Τέλος, κατά την επιστροφή της αναδρομής, καλείται η balance για να εξασφαλιστεί ότι το δέντρο παραμένει ισοζυγισμένο.

```

1 Node* deleteNode(Node* root, int date[3]) {
2     if(!root) {

```



```

3         printf("No nodes present.\n");
4         return NULL;
5     }
6
7     int cmp = compareDates(date, root->record->date);
8
9     if (cmp < 0) {
10         root->left = deleteNode(root->left, date);
11     } else if (cmp > 0) {
12         root->right = deleteNode(root->right, date);
13     } else {
14         if(!root->left || !root->right) {
15             Node* temp = root->left ? root->left : root->right;
16             free(root);
17             return temp;
18         }
19
20         Node* junior = minNode(root->right);
21         swapNodes(root, junior);
22         root->right = deleteNode(root->right, junior->record->date);
23     }
24
25     return balance(root);
26 }

```

Listing 7: Συνάρτηση deleteNode

5. Για το πέμπτο ερώτημα, απελευθερώνεται η μνήμη του δυναμικού πίνακα δεδομένων (free(original)) αποτρέποντας διαρροές μνήμης (memory leaks) και το πρόγραμμα τερματίζει.

6 Άσκηση (B): Υλοποίηση Εφαρμογής με BST, με βάση CUMULATIVE

6.1 Περιγραφή

Το Δέντρο AVL διατάσσεται με βάση την τιμή Cumulative. πειδή στα δεδομένα είναι βέβαιο ότι πολλές διαφορετικές ημέρες θα έχουν ακριβώς την ίδια τιμή Cumulative, η κλασική δομή του κόμβου επεκτάθηκε.

Αντί να δημιουργούνται περιττοί κόμβοι στο δέντρο που θα αλλοίωναν την ισορροπία του, ενσωματώθηκε μια απλά συνδεδεμένη λίστα (Singly Linked List) στο εσωτερικό κάθε κόμβου μέσω του δείκτη EqNode* equal.

Όταν η συνάρτηση insert εντοπίσει ότι το Cumulative της νέας εγγραφής ισούται με ένα ήδη υπάρχον (cmp == 0), δεν αλλοιώνει το δέντρο, αλλά καλεί την insertEqNode_End η οποία προσθέτει την εγγραφή στο τέλος της λίστας του συγκεκριμένου κόμβου.

```

1     typedef struct EqNode {
2         Record* value;
3         struct EqNode* next;
4     } EqNode;
5
6     typedef struct Node {
7         Record* record;
8         struct Node* left;
9         struct Node* right;

```

```

10     int height;
11
12     EqNode* equal;
13 } Node;

```

Listing 8: Δομή κόμβου

1. Για το πρώτο ερώτημα, αξιοποιείται η ιδιότητα του Δυαδικού Δέντρου Αναζήτησης ότι η ελάχιστη τιμή βρίσκεται στο αριστερότερο φύλλο. Η συνάρτηση findMin ξεκινά από τη ρίζα και μεταβαίνει διαρκώς στο αριστερό παιδί (root = root->left) μέχρι να συναντήσει NULL.

```

1 Node* findMin(Node* root) {
2     if(!root) return NULL;
3
4     while(root->left) {
5         root = root->left;
6     }
7     return root;
8 }

```

Listing 9: Συνάρτηση findMin

Μόλις βρεθεί ο κόμβος, καλείται η printEqList η οποία διατρέχει τη συνδεδεμένη λίστα του κόμβου και εκτυπώνει όλες τις ημερομηνίες που μοιράζονται αυτή την ελάχιστη τιμή.

```

1 void printDates_Min(Node* root) {
2     Node* min = findMin(root);
3
4     if (!min) {
5         printf("The tree is empty.\n");
6         return;
7     }
8
9     printf("The minimum Cumulative is: %lld", min->record->cumulative);
10    printEqList(min);
11 }

```

Listing 10: Συνάρτηση printDates_Min

```

1 void printEqList(Node* node) {
2     if(!node) return;
3
4     Record* r = node->record;
5     printf("Cumulative: %lld\n \tDate: %d/%d/%d\n", r->cumulative, r->date[0],
6         r->date[1], r->date[2]);
7
8     EqNode* eq = node->equal;
9     while(eq!= NULL) {
10        r = eq->value;
11
12        printf("Cumulative: %lld\n \tDate: %d/%d/%d\n", r->cumulative, r->date
13    [0],
14        r->date[1], r->date[2]);
15        eq = eq->next;
16    }
17 }

```

Listing 11: Συνάρτηση printEqList

2. Για το δεύτερο ερώτημα, η συνάρτηση findMax διασχίζει το δέντρο ακολουθώντας συνεχώς τον δείκτη προς το δεξί παιδί ($root = root \rightarrow right$) μέχρι να βρει τον τελευταίο κόμβο. Και πάλι χρησιμοποιείται η printEqList για να εκτυπωθούν όλες οι ισοβαθμούσες ημερομηνίες.

```

1 Node* findMax(Node* root) {
2     if(!root) return NULL;
3
4     while(root->right) {
5         root = root->right;
6     }
7     return root;
8 }

```

Listing 12: Συνάρτηση findMax

```

1 void printDates_Max(Node* root) {
2     Node* max = findMax(root);
3
4     if (!max) {
5         printf("The tree is empty.\n");
6         return;
7     }
8
9     printf("The maximum Cumulative is: %lld\n", max->record->cumulative);
10    printEqList(max);
11 }

```

Listing 13: Συνάρτηση printDates_Max

7 Άσκηση (Γ): Υλοποίηση Εφαρμογής με Hashing με αλυσίδες, με βάση DATE

7.1 Περιγραφή

Υλοποιήθηκε με Hash Table χρησιμοποιώντας την μέθοδο αλυσίδων για την αντιμετώπιση των συγκρούσεων.

Η δομή HashMap αποτελείται από έναν πίνακα δεικτών μεγέθους m .

```

1 typedef struct {
2     Node **buckets;
3     int m;
4     int totalRecords;
5 }HashMap;

```

Listing 14: Δομή HashMap

Διατηρείται η ίδια αποδοτική διαχείριση μνήμης με τα προηγούμενα ερωτήματα: Κάθε κόμβος της αλυσίδας δεν αντιγράφει τα δεδομένα, αλλά περιέχει απλώς έναν δείκτη προς τον κεντρικό πίνακα original.

Η συνάρτηση hashing υπολογίζει το άθροισμα των ASCII κωδικών των χαρακτήρων του αλφαριθμητικού της ημερομηνίας. Το τελικό index (κάδος) προκύπτει από το υπόλοιπο της διαίρεσης αυτού του αθροίσματος με το μέγεθος του πίνακα m : $Index = (\sum_i ASCII(Date[i])) \pmod m$

Για την αντιμετώπιση συγκρούσεων όταν μια εγγραφή πρέπει να μπει σε έναν κάδο, ο νέος κόμβος εισάγεται πάντα στην αρχή (head) της συνδεδεμένης λίστας (newNode->next = map->buckets[index]).

1. Για το πρώτο ερώτημα, υπολογίζεται το index της δοθείσας ημερομηνίας μέσω της hash_date. Στη συνέχεια, ο αλγόριθμος διασχίζει μόνο τη συνδεδεμένη λίστα του συγκεκριμένου κάδου, συγκρίνοντας τα strings (strcmp). Επειδή υπάρχουν διπλότυπες ημερομηνίες, η αναζήτηση δεν σταματάει στο πρώτο ταίριασμα, αλλά συνεχίζει για να εμφανίσει όλες τις σχετικές εγγραφές.

```
1 int hash_date(char* date, int m) {
2     unsigned long sum = 0;
3     for(int i = 0; i < DATE-1; i++) {
4         sum += date[i];
5     }
6     return (int)(sum % (unsigned long)m);
7 }
```

Listing 15: Συνάρτηση hash_date

```
1 int searchByDate(HashMap* map, char* date) {
2     int index = hash_date(date, map->m);
3     Node *now = map->buckets[index];
4     bool found = false;
5
6     printf("\n===== Results for date: %s =====\n", date);
7
8     while(now) {
9         if(strcmp(now->record->date, date) == 0) {
10             printf("\tCumulative: %lld\n", now->record->cumulative);
11             found = true;
12         }
13         now = now->next;
14     }
15
16     if(!found) {
17         printf("[!] No Record in this date: %s\n", date);
18     }
19 }
```

Listing 16: Συνάρτηση searchByDate

2. Για το δεύτερο ερώτημα, η τροποποίηση λειτουργεί ακριβώς όπως η αναζήτηση. Μόλις εντοπιστεί η ημερομηνία μέσα στην αλυσίδα του κάδου, ο χρήστης εισάγει τη νέα τιμή και αυτή ενημερώνεται άμεσα μέσω του δείκτη. Η διαδικασία επαναλαμβάνεται για όλες τις εγγραφές της ίδιας ημερομηνίας.

```
1 void modifyByDate(HashMap* map, char* date) {
2     int index = hash_date(date, map->m);
3     Node *now = map->buckets[index];
4     bool found = false;
5     long long newCumulative;
6
7     while(now) {
8         if(strcmp(now->record->date, date) == 0) {
9             printf("Current record : \t Cumulative: %lld\n", now->record->
10 cumulative);
```

```

11     printf("New Cumulative: ");
12     if (scanf("%lld", &newCumulative) != 1) {
13         printf("Value not acceptable.\n");
14         while(getchar() != '\n');
15         now = now->next;
16         continue;
17     }
18
19     now->record->cumulative = newCumulative;
20     printf("Value updated : \tCumulative = %lld\n", newCumulative);
21     found = true;
22 }
23 now = now->next;
24 }
25
26 if(!found) {
27     printf("[!] No Record in this date: %s\n",date);
28 }
29 }

```

Listing 17: modifyByDate

3. Για το τρίτο ερώτημα, υπολογίζεται το index και ξεκινά η διάσχιση της λίστας του κάδου χρησιμοποιώντας δύο δείκτες (now και before). Αν είναι ο πρώτος κόμβος (δεν υπάρχει before), η κεφαλή του κάδου δείχνει στον επόμενο (map->buckets[index] = now->next). Αν βρίσκεται ενδιάμεσα, παρακάμπτεται συνδέοντας τον προηγούμενο με τον επόμενο (before->next = now->next). Στη συνέχεια απελευθερώνεται η μνήμη του κόμβου (free(deleting)). Ο αλγόριθμος συνεχίζει μέχρι το τέλος της λίστας για να διαγράψει όλα τα διπλότυπα.

```

1 void deleteByDate(HashMap* map, char* date) {
2     int index = hash_date(date, map->m);
3     Node *now = map->buckets[index];
4     Node *before = NULL;
5     bool found = false;
6
7     while(now) {
8         if(strcmp(now->record->date, date) == 0) {
9             Node* deleting = now;
10            if(before) {
11                before->next = now->next;
12            } else {
13                map->buckets[index] = now->next;
14            }
15
16            now = now->next;
17            free(deleting);
18            map->totalRecords--;
19            found = true;
20        } else {
21            before = now;
22            now = now->next;
23        }
24    }
25
26    if(!found) {
27        printf("[!] No Record in this date: %s\n",date);
28    }
29 }

```

```

29     }
30 }

```

Listing 18: deleteByDate

4. Για το τέταρτο ερώτημα, καλείται η συνάρτηση `destroyMap` η οποία διατρέχει όλους τους m κάδους και διαγράφει σειριακά όλους τους κόμβους (`free(now)`) κάθε αλυσίδας. Απελευθερώνεται ο πίνακας των κάδων και ο κεντρικός πίνακας `original` και τερματίζει.

8 Συμπεράσματα

8.1 Ταξινόμηση και Εύρος

Το Δέντρο AVL υπερέχει όταν απαιτείται ταξινόμηση (In-order) ή εύρεση ακραίων τιμών (Min/Max), καθώς το επιτυγχάνει σε χρόνο $O(\log n)$. Αντίθετα, το Hash Table δεν έχει έννοια διάταξης, καθιστώντας τέτοια ερωτήματα πολύ αργά ($O(n)$).

8.2 Ταχύτητα Αναζήτησης Συγκεκριμένου Κλειδιού

Για την εύρεση μιας ακριβούς ημερομηνίας, το Hash Table είναι θεωρητικά η ταχύτερη επιλογή με μέση πολυπλοκότητα $O(1)$, θυσιάζοντας όμως την ταξινόμηση.

8.3 Μοναδικότητα Δεδομένων

Η διαχείριση του B μέρους ανέδειξε πώς ένα Δέντρο Αναζήτησης μπορεί να παραμείνει ταχύτατο χρησιμοποιώντας εσωτερικές συνδεδεμένες λίστες όταν το κλειδί αναζήτησης δεν είναι μοναδικό.

9 Παράρτημα

9.1 Πηγαίος Κώδικας Main

```

1  int main(){
2
3      int choice = 0;
4
5      while(choice!=3) {
6
7          choice = printMenuBSTorHashing();
8
9          switch(choice) {
10             case 1:
11
12                 choice = printMenuBST_datecumulative();
13
14                 switch(choice) {
15                     case 1:
16                         runBST_date();
17                     break;
18                     case 2:
19                         runBST_cumulative();
20                     break;

```

```

21         default:
22             printf("Invalid choice.\n");
23     }
24     break;
25     case 2:
26         runHashing();
27         break;
28     case 3:
29         printf("Ending program. . .\n");
30         break;
31     default:
32         printf("Invalid choice.\n");
33 }
34 }
35
36 return 0;
37 }

```

Listing 19: Main

9.2 Στιγμιότυπα Οθόνης

Παρακάτω παρουσιάζονται μερικά παραδείγματα:

```

=====MENU A=====
1 => BST
2 => Hashing
3 => Exit
Choice: 2

=====MENU=====
1 => Search Cumulative by Date
2 => Modify Cumulative by Date
3 => Delete Record by Date
4 => Exit
Choice: 3
Enter the date (e.g. 13/03/2018): 12/10/2017

=====MENU=====
1 => Search Cumulative by Date
2 => Modify Cumulative by Date
3 => Delete Record by Date
4 => Exit
Choice: 1
Enter the date (e.g. 13/03/2018): 12/10/2017

===== Results for date: 12/10/2017 =====
[!] No Record in this date: 12/10/2017

```

```

=====MENU A=====
1 => BST
2 => Hashing
3 => Exit
Choice: 1

=====MENU A=====
1 => BST by Date
2 => BST by Cumulative
3 => Exit
Choice: 1

=====MENU A=====
1 => In-order Display
2 => Search by date
3 => Modify cumulative
4 => Delete by date
5 => Exit
Choice: 2
Enter date (DD/MM/YYYY): 12/10/2019
Cumulative: 45972000000

=====MENU A=====
1 => In-order Display
2 => Search by date
3 => Modify cumulative
4 => Delete by date
5 => Exit
Choice: 

```

```

=====MENU A=====
1 => BST
2 => Hashing
3 => Exit
Choice: 1

=====MENU A=====
1 => BST by Date
2 => BST by Cumulative
3 => Exit
Choice: 2

=====MENU=====
1 => Display Minimum Cumulative Dates
2 => Display Maximum Cumulative Dates
3 => Exit
Choice: 1
The minimum Cumulative is: 104000000Cumulative: 104000000
Date: 1/1/2015

=====MENU=====
1 => Display Minimum Cumulative Dates
2 => Display Maximum Cumulative Dates
3 => Exit
Choice: 

```